

Cahier des Charges

Application web de simulation de capteurs IoT

1. Contexte et Objectifs

L'objectif de ce projet est de développer une application web qui permet de simuler des capteurs IoT mesurant différentes grandeurs physiques (telles que définies dans la RFC 8798). Cette application sera développée en Python en utilisant le framework Flask, et sera déployée dans un environnement Docker. Les utilisateurs pourront gérer les capteurs via un système CRUD (Create, Read, Update, Delete), et envoyer les valeurs simulées à un broker MQTT de manière flexible (depuis un fichier CSV, aléatoirement, ou via une interface utilisateur interactive).

2. Fonctionnalités Principales

2.1 Gestion des Capteurs (CRUD)

- **Création de capteurs :**
 - Formulaire pour définir le nom du capteur, un UID généré & unique, la grandeur physique (conforme à la RFC 8798), l'unité de mesure, la précision, et la fréquence d'échantillonnage (envoi des valeurs), etc... (liste à compléter).
- **Lecture des capteurs :**
 - Vue liste pour afficher tous les capteurs enregistrés avec possibilité de filtrer, trier, et rechercher.
- **Mise à jour des capteurs :**
 - Possibilité de modifier les propriétés d'un capteur existant.
- **Suppression de capteurs :**
 - Option de suppression d'un capteur de la base de données.

2.2 Simulation de valeurs

- **Envoi de valeurs simulées à un broker MQTT :**
 - Interface pour configurer le broker MQTT (adresse, port, topic, etc.).
- **Méthodes de simulation des valeurs :**
 - **Import depuis un fichier CSV :** Permettre de télécharger un fichier CSV contenant des données de capteurs préétablies.
 - **Génération aléatoire :** Génération de valeurs aléatoires pour les capteurs en fonction de la plage définie.
 - **Interface de réglage manuel :** Slider pour régler manuellement la valeur des capteurs avec un pas de temps donné.

2.3 Envoi Automatisé des Données

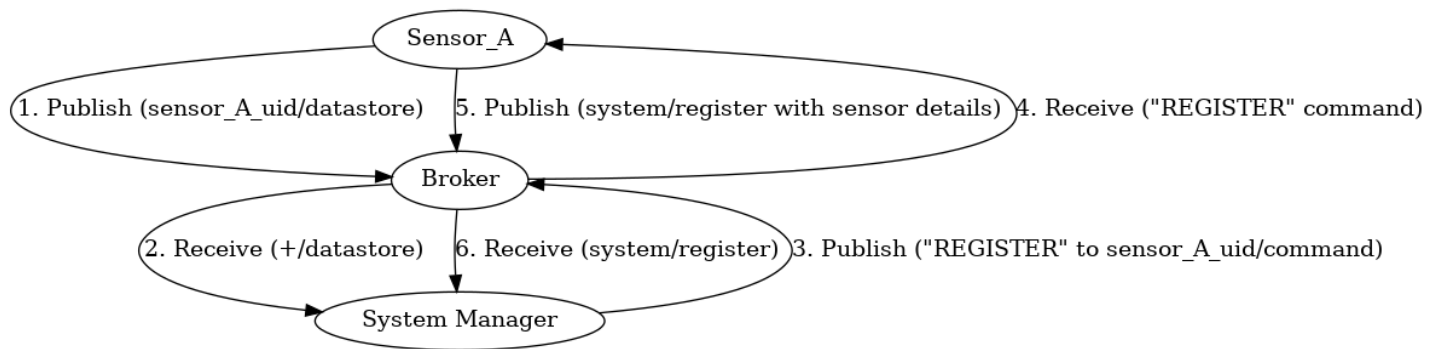
- **Planification de l'envoi des données :**

- Intégration d'un système de planification (cron job ou équivalent en Python) pour envoyer automatiquement les données des capteurs simulés à intervalles réguliers.
- La possibilité de configurer la fréquence d'envoi (par exemple, toutes les 5 secondes, toutes les minutes, toutes les heures) viens des spécifications du capteur cf. 2.1

2.4 Appairage automatique / Handshaking a un système connexe

Le système utilise un broker MQTT et se compose de plusieurs capteurs (ou clients) et d'un gestionnaire de système. Chaque capteur publie ses données sur un topic nommé `datastore`. Si un capteur n'est pas reconnu (son uid n'est pas dans la liste des capteurs connus), le gestionnaire de système publie un message sur le topic `command` pour indiquer la nécessité d'un enregistrement du capteur. Ensuite, le capteur répond sur un topic `register` avec ses caractéristiques détaillées.

ATTENTION: Tout les étapes ici ne sont données que pour illustrer la méthode et le projet n'intégrera pas le gestionnaire de système et de fait seule les étapes 4 et 5 seront à gérer.



Étapes du Processus de Handshaking :

1. **Publication de Données :**

Un capteur (ex: `sensor_A`) publie une donnée sur le topic `sensor_A_uid/datastore`.

2. **Vérification par le Système :**

Le gestionnaire de système est abonné au topic `+/datastore` (utilise le caractère générique `+`, qui correspond à n'importe quel niveau unique dans la hiérarchie des topics).

Lorsqu'il reçoit une publication sur ce topic, il vérifie si l'uid du capteur est dans la liste des capteurs connus.

3. **Capteur Inconnu :**

Si le nom du capteur n'est pas connu, le système publie le message **"REGISTER"** sur le topic `sensor_A_uid/command`.

4. **Demande d'Enregistrement :**

Ce message demande au capteur de s'enregistrer ; en publiant un message sur le topic `system/register`

5. **Enregistrement du Capteur :**

Le capteur répond sur le topic `system/register` avec un JSON contenant toutes ses caractéristiques.

Exemple :

```
{ "uid": "sensor_A_uid", "description": "Temperature", "unit": "Celsius",  
"min_value": "-40", "max_value": "85", "delta_value": "0.1", "period": "60",  
"min_period": "30", "max_period": "600", "read_only": "false" }
```

6. Mise à Jour du Système :

Le gestionnaire de système ajoute à la liste des capteurs connus et enregistre les caractéristiques pour une future utilisation.

2.5 Interface Utilisateur

- Interface web intuitive développée en Flask avec des vues HTML/CSS et JavaScript pour une interaction dynamique.
- Tableau de bord pour visualiser l'état des capteurs et les valeurs envoyées.

3. Spécifications Techniques

3.1 Technologies

- **Backend** : Flask (Python)
- **Frontend** : HTML, CSS, JavaScript (avec éventuellement le framework Vue.js)
- **Base de données** : SQL pour stocker les informations des capteurs
- **MQTT** : Paho-MQTT (bibliothèque Python) pour la communication avec le broker MQTT
- **Conteneurisation** : Docker pour le déploiement et la gestion des conteneurs

3.2 Déploiement

- Dockerfile pour construire l'image Docker de l'application Flask
- Docker Compose pour orchestrer les services (application Flask, base de données, broker MQTT, service de planification si nécessaire)
- Configuration pour le déploiement sur un serveur cloud (ex. AWS, Azure) ou sur une infrastructure locale

3.3 Sécurité

- Authentification des utilisateurs (gestion des rôles : administrateur, utilisateur standard)
- Chiffrement des communications avec le broker MQTT (TLS)
- Protection contre les injections SQL, XSS, CSRF

3.4 Tests et Validation

- Tests unitaires pour toutes les fonctions de l'application (Python unittest ou pytest)
- Tests de performance pour la génération et l'envoi de données vers le broker MQTT
- Tests de sécurité (scans de vulnérabilités, analyse de code statique)

4. Exigences Non Fonctionnelles

- **Performance** : L'application doit supporter au moins 100 capteurs actifs envoyant des données à une fréquence de n seconde. (A déterminer)
- **Fiabilité** : Assurer que les tâches planifiées sont exécutées de manière fiable, même en cas de redémarrage du conteneur ou de l'application.
- **Flexibilité** : Permettre la modification dynamique des tâches planifiées sans avoir besoin de redémarrer l'application ou les conteneurs.
- **Scalabilité** : Possibilité d'ajouter de nouveaux capteurs et d'augmenter la fréquence d'échantillonnage sans dégradation significative des performances. (A estimer)
- **Maintenabilité** : Code bien structuré et commenté pour faciliter la maintenance et les futures évolutions.
- **Portabilité** : Application Dockerisée pour un déploiement facile sur différentes plateformes.

5. Livrables

- Code source de l'application Flask
- Fichiers Dockerfile et Docker Compose
- Documentation utilisateur (guide de configuration et d'utilisation)
- Documentation technique (architecture, API, installation)
- Scripts de test et rapport de validation

6. Suivi et Communication

- Réunions de suivi d'avancement
- Utilisation d'un outil de gestion de projet (ex. tableau Kanban) pour suivre les tâches et les priorités
- Git